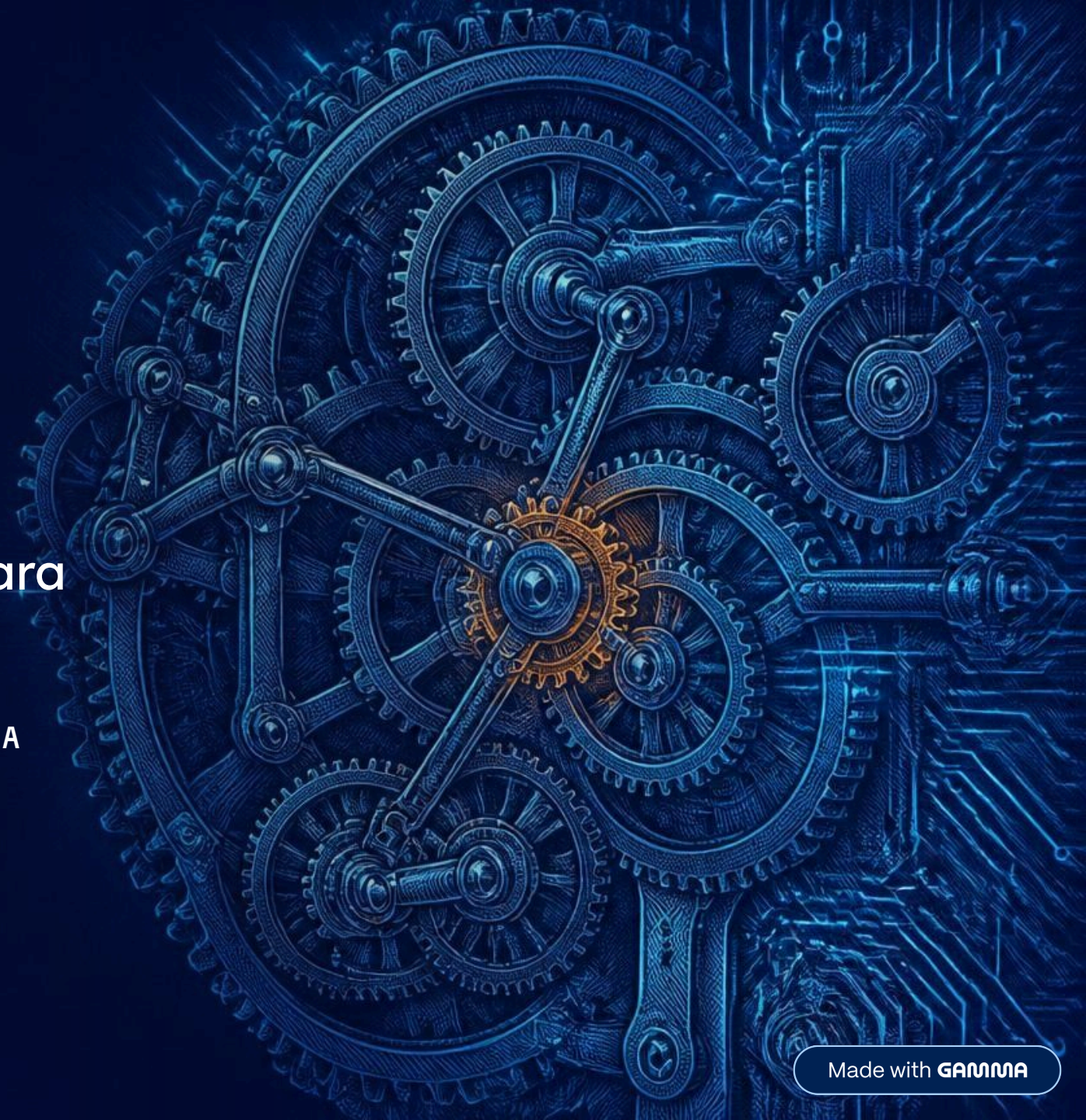
Python: ingeniería y software

Fundamentos, arquitectura de ejecución y diseño idiomático para construir sistemas robustos.

ARQUITECTURA DE SOFTWARE · GUÍA TÉCNICA



Made with GAMMA

DEFINICIÓN Y PRINCIPIOS

Qué es Python

Lenguaje interpretado

De alto nivel y tipado dinámico.

Multiparadigma

Orientado a objetos, imperativo y funcional.

Sintaxis legible

La indentación define los bloques.

Filosofía Python

“Baterías incluidas” y el Zen de Python.

Made with GAMMA

DOMINIOS INDUSTRIALES

Python en el ecosistema de ingeniería

Datos y cómputo científico

NumPy, pandas, SciPy

Automatización, scripting y DevOps

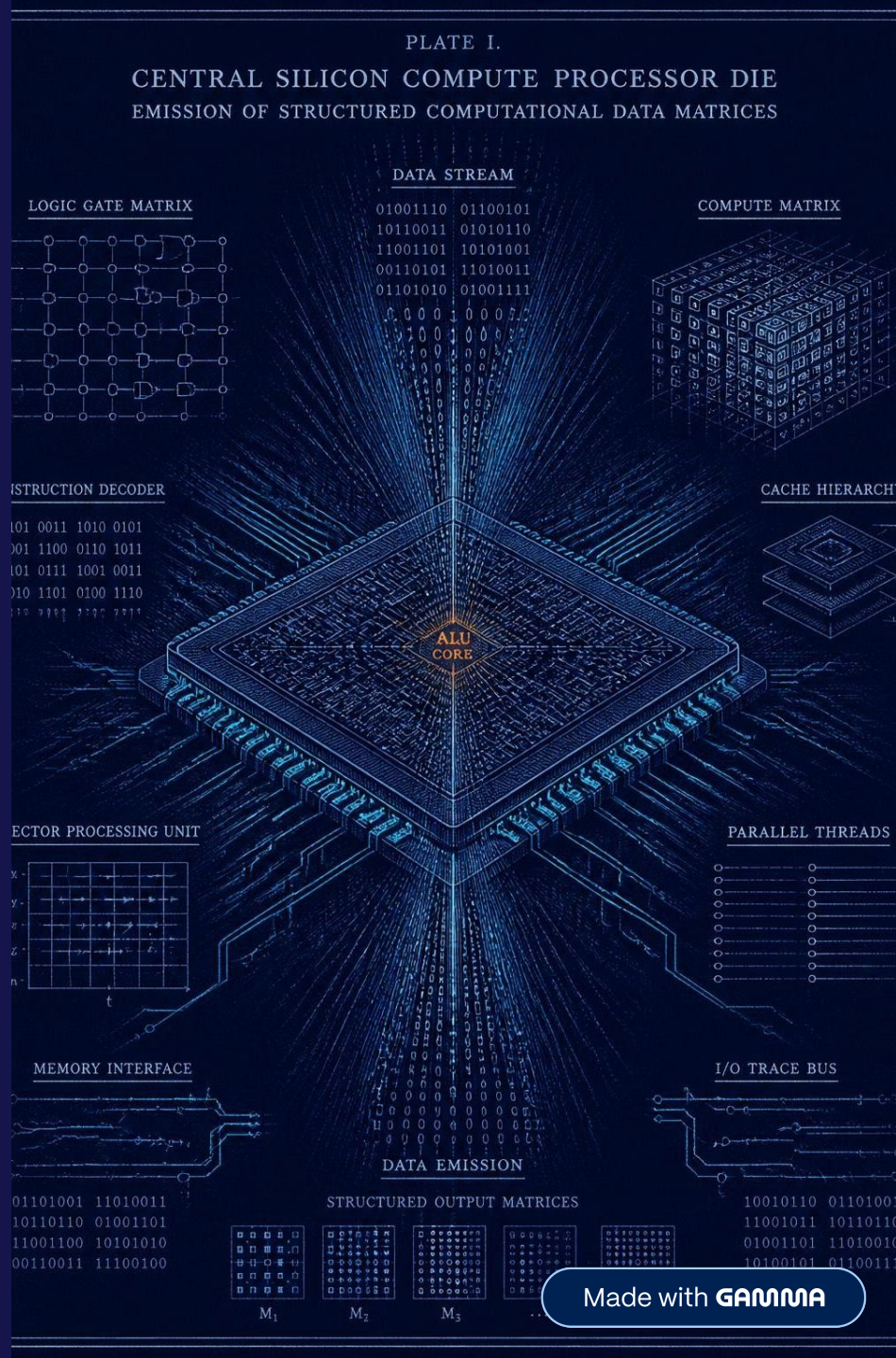
Automatización y scripting

Backend web

Django, FastAPI

Machine learning

PyTorch, scikit-learn



Made with GAMMA



Historia: de un hobby navideño a estándar global

1989

Guido van Rossum inicia el proyecto en el CWI, Países Bajos.

1991

Primera versión pública, 0.9.0.

2000 · Python 2.0

Listas por comprensión y recolector de ciclos.

2008 · Python 3.0

Ruptura de compatibilidad.

Made with GAMMA

MODERNIZACIÓN Y GOBERNANZA

Python 2 vs Python 3 y la era moderna

TRANSICIÓN

Del legado al nuevo modelo

- Python 2 llegó a fin de vida en 2020.
- Gobernanza: de BDFL a Steering Council desde 2018.

ERA MODERNA

Lenguaje en evolución

- Type hints, f-strings, asyncio y dataclasses.
- Ciclo anual de versiones y mejoras de rendimiento recientes.

Made with GAMMA

SISTEMA DE TIPOS

Concepto clave: modelo de datos y tipos

OBJETOS Y MUTABILIDAD

Todo es un objeto; las variables son referencias.

Mutables (list, dict, set) frente a inmutables (tuple, str, int).

a = [1]; b = a; b.append(2) modifica el objeto compartido.

DUCK TYPING Y PROTOCOLOS

Duck typing: importa el comportamiento que ofrece un objeto.

Los protocolos usan métodos especiales (__dunder__), como __add__ o __getitem__.

Made with GAMMA

EVALUACIÓN PEREZOSA

Iteradores, generadores y comprensiones

Procesamiento de flujos masivos de datos manteniendo la complejidad espacial en O(1).

Protocolo de iteración

Cualquier objeto con `__iter__()` que retorna un iterable y `__next__()` que arroja `StopIteration` al agotarse.

Generadores con yield

Pausan el frame de ejecución y liberan CPU/memoria entre llamadas sucesivas sin almacenar el dataset completo.

Comprensiones idiomáticas

Sintaxis declarativa compacta para listas, conjuntos y diccionarios con bucle y filtro interno en bytecode optimizado.

STREAMING CON MEMORIA CONSTANTE

```
def stream_metrics(file_path: str):
with open(file_path) as f:
for line in f:
if line.startswith("METRIC:"):
yield float(line.split()[1])
```

COMPRENSIONES DECLARATIVAS

```
# Diccionario filtrado con set comprehension previa

valid_ids = {node.id for node in cluster if node.active}
scores = {k: v * 1.5 for k, v in raw_data.items() if k in
valid_ids}
```

Made with GAMMA

REUTILIZACIÓN E INGENIERÍA

Funciones, decoradores y gestores de contexto

Funciones como objetos de primera clase y closures: patrones para separar responsabilidades y liberar recursos con seguridad.

DECORADORES (WRAPPERS)

@wraps(func) · wrapper(*args, **kwargs)

Logging, caché y validación sin modificar la lógica base.

CONTEXT MANAGERS (PEP 343)

with recurso: · try: yield · finally: liberar()

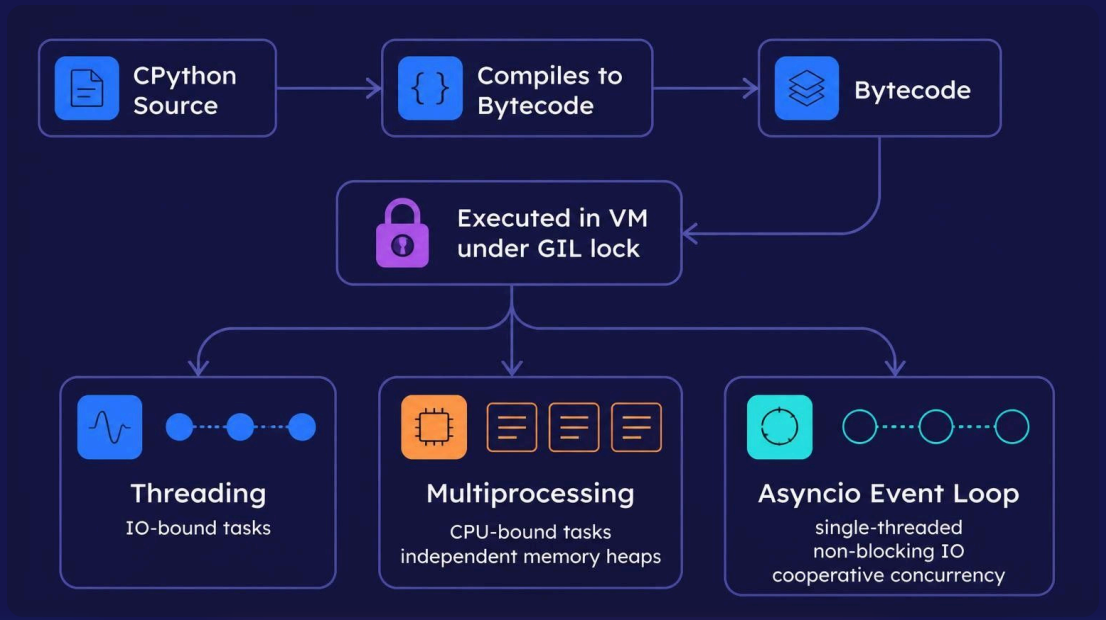
Liberan recursos con seguridad mediante __enter__ y __exit__.

Made with GAMMA

INTERNOS DEL RUNTIME

Ejecución, el GIL y modelos de concurrencia

Elegir la estrategia adecuada requiere comprender cómo CPython interactúa con los núcleos del procesador.



THREADING

I/O Concurrente

El GIL se libera en llamadas syscall de red o disco. Hilos livianos del OS para sockets y queries.

MULTIPROCESSING

Paralelismo CPU

Procesos aislados con heap propio; evade el GIL por completo a expensas de costo de IPC y serialización.

ASYNCIO

Event Loop Único

Concurrencia cooperativa de decenas de miles de conexiones simultáneas sin overhead de context switch.

Made with GAMMA

BUENAS PRÁCTICAS

Tips de ingeniería para código robusto



Entornos y dependencias

Usa entornos virtuales y fija dependencias con `venv`, `pip` y `poetry`.



Tipado, formato y pruebas

Type hints con `mypy`; formato con `ruff` o `black`; pruebas con `pytest`.



Evita antipatrones

Evita argumentos por defecto mutables y los `except` desnudos.



Perfila antes de optimizar

Usa `cProfile`, vectorización y extensiones en C.

Made with GAMMA